Fully Coupled 1-D Schrodinger–Poisson Solver for a (100) Si MOS Inversion Layer

Including $p(\phi)$, $N_A^-(\phi)$, a Three-Subband Reduced Model, and t_{ox}–N_A QM-Onset Maps

Numerical derivation and implementation note

Abstract

This note replaces the earlier strong-inversion approximation $p \approx 0$ by a nonlinear Poisson equation that retains holes, incomplete acceptor ionization, and the quantum electron density. The quantum density is obtained from two (100)-Si valley families with Fermi–Dirac subband occupation. The electrostatic potential is determined by a MOS oxide/gate Robin boundary condition at the interface and a bulk Dirichlet condition. Python and MATLAB implementations are included. The numerical example follows the benchmark conditions used by Janik and Majkusiak: $N_A = 10^{17}\,\mathrm{cm}^{-3}$, $t_{ox} = 3\,\mathrm{nm}$, $T = 300\,\mathrm{K}$, and $V_G = 4\,\mathrm{V}$, while the flat-band voltage remains an explicit model parameter because the paper does not fully specify the absolute gate/oxide/Si band alignment.

Nomenclature

x	Distance normal to the Si/SiO_2 interface
ϕ	Electrostatic potential in silicon
ϕ_s	Surface electrostatic potential, $\phi(0)$
ϕ_F	Bulk Fermi potential used in the threshold criterion
E_c, E_v	Conduction- and valence-band edges
E_F	Equilibrium Fermi level
E_{vj}	Energy of subband j in valley family v
ψ_{vj}	Normalized envelope wavefunction
n_{QM}	Quantum electron concentration
p	Hole concentration
N_A	Total acceptor concentration
N_A^-	Ionized acceptor concentration
N_{vj}	Sheet population of subband (v, j)
N_s	Number of retained quantum subbands per valley family
Q_{inv}	Inversion-layer sheet charge
$\bar{x}$	Inversion-charge centroid measured from the interface
$n_3(x)$	Electron density from the three-subband reduced model
$n_{10}(x)$	Ten-state-per-valley numerical reference density
g_v	Valley degeneracy
$m^*_{z,v}$	Confinement effective mass
$m^*_{d,v}$	In-plane density-of-states mass
m_l, m_t	Longitudinal and transverse Si masses
q	Magnitude of the electronic charge
k_B	Boltzmann constant
T	Absolute temperature
V_T	Thermal voltage, k_BT/q
V_{th}	MOS threshold voltage (kept distinct from thermal voltage V_T)
ε_{Si}	Silicon permittivity
ε_{ox}	Oxide permittivity
C_{ox}	Oxide capacitance per unit area
t_{ox}	Oxide thickness
V_G	Applied gate voltage
V_{FB}	Flat-band voltage
L	Silicon simulation-domain depth
Δx	Spatial mesh spacing
H_v	Finite-difference Hamiltonian for valley v
R_i	Nonlinear Poisson residual at node i

1 Physical model

For a planar MOS structure, confinement is along x, normal to the Si/SiO_2 interface. The electron envelope function in valley family v and subband j obeys

$$-\frac{\hbar^2}{2m^*_{z,v}}\frac{d^2\psi_{vj}}{dx^2} + E_c(x)\psi_{vj}(x) = E_{vj}\psi_{vj}(x), \tag{1}$$

with $E_c(x) - E_{c,\mathrm{bulk}} = -q\phi(x)$. For (100) Si there are two valley families:

Family	Degeneracy g_v	confinement mass $m_{z,v}$	in-plane DOS mass $m_{d,v}$
L	2	m_l	m_t
T	4	m_t	$\sqrt{m_l m_t}$

The numerical values used here are $m_l = 0.9163m_0$ and $m_t = 0.2245m_0$.

1.1 Fermi–Dirac subband population

The sheet population of subband (v, j) is

$$N_{vj} = \frac{g_v m^*_{d,v} k_B T}{\pi\hbar^2} \ln\left[1 + \exp\left(\frac{E_F - E_{vj}}{k_B T}\right)\right]. \tag{2}$$

The quantum electron concentration is therefore

$$\boxed{n_{\mathrm{QM}}(x) = \sum_{v,j} N_{vj}|\psi_{vj}(x)|^2 + n_0.} \tag{3}$$

The small n_0 term restores the correct equilibrium minority-electron floor in the far bulk.

2 Three-subband reduced model

A reduced model is obtained by retaining only the lowest three confined states in each valley family,

$$\boxed{n_3(x) = \sum_{v\in\{L,T\}}\sum_{j=1}^{3} N_{vj}|\psi_{vj}(x)|^2 + n_0.} \tag{4}$$

The reference calculation used in this report retains ten finite-box eigenstates per valley family,

$$n_{10}(x) = \sum_{v\in\{L,T\}}\sum_{j=1}^{10} N_{vj}|\psi_{vj}(x)|^2 + n_0. \tag{5}$$

Both models are solved *self-consistently*: changing the number of retained subbands changes the electron density, which changes Poisson's equation, which in turn changes the potential and the eigenstates. Thus the comparison is not a post-processing truncation of one fixed potential.

Spinelli, Benvenuti, and Pacelli used a related computational reduction: three low subbands were treated quantum mechanically and higher-energy states were represented by a classical tail. The present "3-subband" comparison intentionally isolates the effect of truncating the discrete spectrum; it does not add their separate classical-tail correction. This distinction is important when interpreting the numerical error.

Define the sheet inversion population of a model M as

$$N_{\mathrm{inv}}^{(M)} = \int_0^L [n_M(x) - n_0]\, dx, \tag{6}$$

and the local density error of the three-subband model as

$$\delta_n(x) = 100\, \frac{n_3(x) - n_{10}(x)}{n_{10}(x)} \quad [\%]. \tag{7}$$

3 Full nonlinear Poisson equation

The earlier $p \approx 0$ approximation is removed. The charge density is

$$\rho(x) = q\left[p(x) - n_{\mathrm{QM}}(x) - N_A^-(x)\right], \tag{8}$$

so that

$$\boxed{\frac{d^2\phi}{dx^2} = \frac{q}{\varepsilon_{Si}}\left[n_{\mathrm{QM}}(x) + N_A^-(\phi) - p(\phi)\right].} \tag{9}$$

3.1 Hole concentration

With nondegenerate holes,

$$p(\phi) = p_0 \exp\!\left(-\frac{\phi}{V_T}\right), \qquad V_T = \frac{k_B T}{q}. \tag{10}$$

3.2 Incomplete acceptor ionization

For an acceptor level $E_A = E_V + E_{A0}$ with degeneracy factor g_A,

$$\boxed{N_A^-(\phi) = \frac{N_A}{1 + g_A \exp\left[\frac{E_A(\phi) - E_F}{k_B T}\right]}.} \tag{11}$$

Since the semiconductor bands shift by $-q\phi$ while the equilibrium Fermi level is fixed,

$$E_F - E_V(x) = (E_F - E_V)_{\mathrm{bulk}} + q\phi(x). \tag{12}$$

Hence, in eV units,

$$N_A^-(\phi) = \frac{N_A}{1 + g_A \exp\left\{\frac{E_{A0} - [(E_F - E_V)_{\mathrm{bulk}} + \phi]}{V_T}\right\}}. \tag{13}$$

The implementation uses $E_{A0} = 45\,\mathrm{meV}$ and $g_A = 4$ as a boron-like incomplete-ionization model. These two quantities are an added modeling choice; they are not specified by the attached Janik–Majkusiak paper.

3.3 Bulk charge neutrality

The bulk Fermi level is obtained from

$$p_0 - n_0 - N_{A0}^- = 0, \tag{14}$$

with

$$p_0 = N_V \exp[-(E_F - E_V)/k_B T], \tag{15}$$

$$n_0 = N_C \exp[-(E_C - E_F)/k_B T]. \tag{16}$$

This ensures that the far-bulk Poisson source tends to zero.

4 MOS boundary conditions

At the Si/SiO_2 interface, Gauss' law through the oxide gives the Robin condition

$$\boxed{\left.\frac{d\phi}{dx}\right|_0 = \frac{C_{ox}}{\varepsilon_{Si}}\left[\phi_s - (V_G - V_{FB})\right],} \qquad C_{ox} = \frac{\varepsilon_{ox}}{t_{ox}}. \tag{17}$$

At the far end of the Si domain,

$$\boxed{\phi(L) = 0.} \tag{18}$$

Thus the surface potential is solved rather than imposed.

5 Finite-difference Hamiltonian

On a uniform mesh $x_i = i\Delta x$,

$$\frac{d^2\psi_i}{dx^2} \approx \frac{\psi_{i+1} - 2\psi_i + \psi_{i-1}}{\Delta x^2}. \tag{19}$$

For valley family v, define

$$t_v = \frac{\hbar^2}{2m^*_{z,v}\Delta x^2}. \tag{20}$$

Then the tridiagonal Hamiltonian is

$$H_v = \begin{bmatrix} U_1 + 2t_v & -t_v & 0 & \cdots \\ -t_v & U_2 + 2t_v & -t_v & \cdots \\ 0 & -t_v & U_3 + 2t_v & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix}, \tag{21}$$

where $U_i = -\phi_i$ when energies are represented in eV relative to the bulk conduction-band edge.

6 Nonlinear Poisson Newton step

For interior node i,

$$R_i = \frac{\phi_{i-1} - 2\phi_i + \phi_{i+1}}{\Delta x^2} - \frac{q}{\varepsilon_{Si}}\left[n_i + N^-_{A,i}(\phi_i) - p_i(\phi_i)\right]. \tag{22}$$

The diagonal Jacobian contribution is

$$\frac{\partial R_i}{\partial \phi_i} = -\frac{2}{\Delta x^2} - \frac{q}{\varepsilon_{Si}}\left[\frac{dN^-_A}{d\phi} - \frac{dp}{d\phi}\right], \tag{23}$$

where

$$\frac{dp}{d\phi} = -\frac{p}{V_T}, \tag{24}$$

$$\frac{dN^-_A}{d\phi} = \frac{N_A}{V_T} f_A(1 - f_A), \qquad f_A = \frac{N^-_A}{N_A}. \tag{25}$$

For the separate classical comparison, $dn/d\phi = n/V_T$ is added to the Jacobian.

7 Self-consistency algorithm

The quantum density is a nonlocal functional of ϕ because changing the potential changes all eigenvalues and eigenfunctions. The implementation therefore uses a stable nested iteration:

1. Start from the fully nonlinear classical Poisson solution.
2. Choose $N_s = 10$ (reference) or $N_s = 3$ (reduced model), solve both valley-family Hamiltonians, and assemble $n_{N_s}(x)$.
3. Solve the nonlinear Poisson equation containing n_{QM}, $p(\phi)$, and $N_A^-(\phi)$ in the same residual.
4. Recompute the quantum density from the updated potential.
5. Mix the quantum density geometrically and repeat until the potential update is small.

No charge term is dropped. The charge mixing is only a numerical stabilization of the nonlocal Schrodinger response.

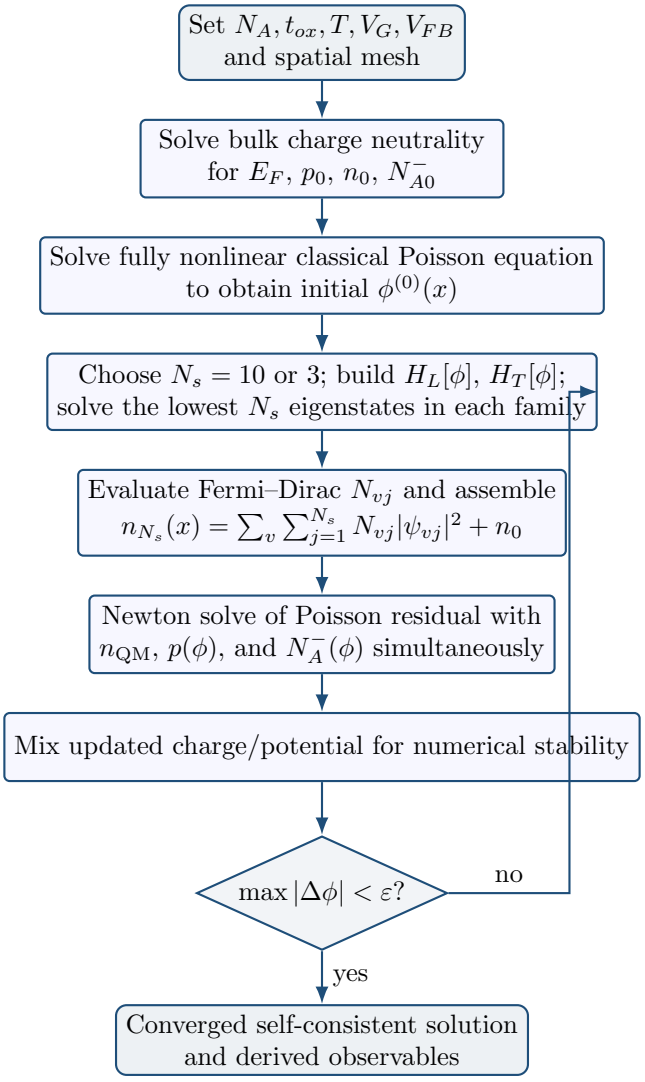


Figure 1: Algorithm flow for either the ten-state reference or the three-subband reduced model. The same nonlinear Poisson physics is used in both cases; only the number of retained eigenstates changes.

8 Numerical result

For the benchmark-style case $N_A = 10^{17}\,\mathrm{cm}^{-3}$, $t_{ox} = 3\,\mathrm{nm}$, $T = 300\,\mathrm{K}$, $V_G = 4\,\mathrm{V}$, and the exposed parameter $V_{FB} = -0.95\,\mathrm{V}$, the Python calculation converged to a quantum surface potential near $1.468\,\mathrm{V}$. The quantum electron-density maximum occurs about $0.70\,\mathrm{nm}$ from the interface. The exact absolute values depend on the chosen flat-band alignment and ionization model, but the displaced quantum peak is the same qualitative behavior shown in Janik and Majkusiak's Fig. 3.

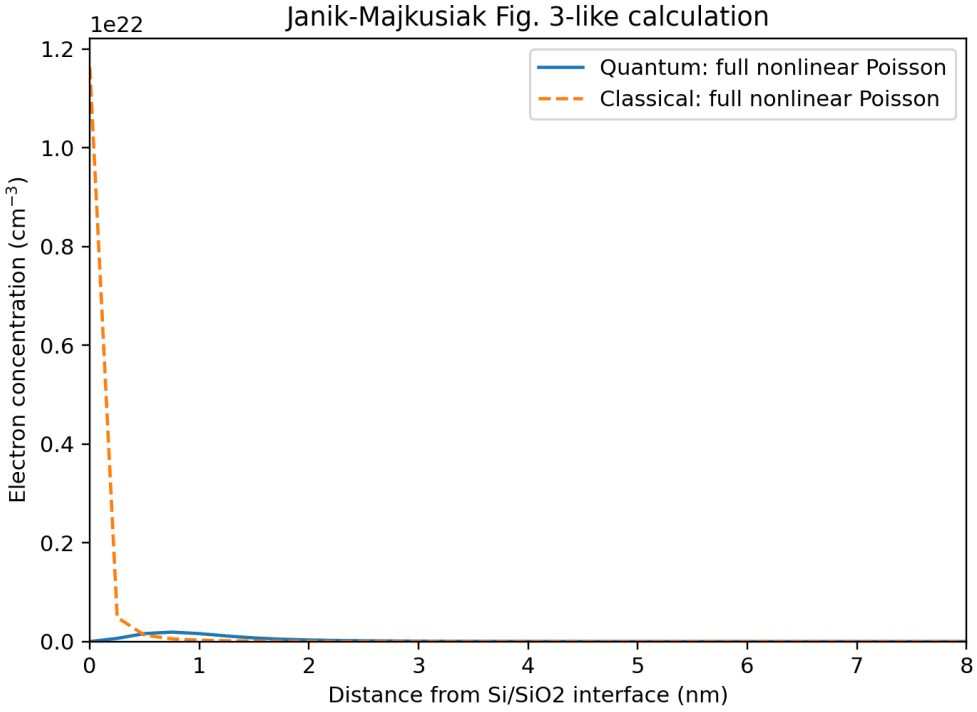


Figure 2: Fully nonlinear classical and quantum electron distributions.

9 Three-subband comparison

For the same benchmark case, the ten-state reference and the three-subband model converge to essentially the same electrostatics. The calculated values are

Model	ϕ_s (V)	$x_{\max}$ (nm)	N_{inv} (cm^{-2})
10 states/valley	1.467962	0.700	2.2537×10^{13}
3 subbands/valley	1.467962	0.700	2.2537×10^{13}

The integrated sheet-density difference is approximately

$$\boxed{\frac{N_{\mathrm{inv}}^{(3)} - N_{\mathrm{inv}}^{(10)}}{N_{\mathrm{inv}}^{(10)}} \times 100\% \simeq -0.0022\%.} \tag{26}$$

For the converged ten-state potential, the first three longitudinal-family states carry about 99.968% of that family's computed subband sheet population, and the first three transverse-family states carry about 99.9992%. Therefore, under this deep-inversion benchmark, higher discrete subbands contribute

negligibly to the total inversion population. This numerical result is specific to the stated bias, doping, domain, and band-alignment assumptions; lower transverse fields or different devices can increase the importance of higher-energy states.

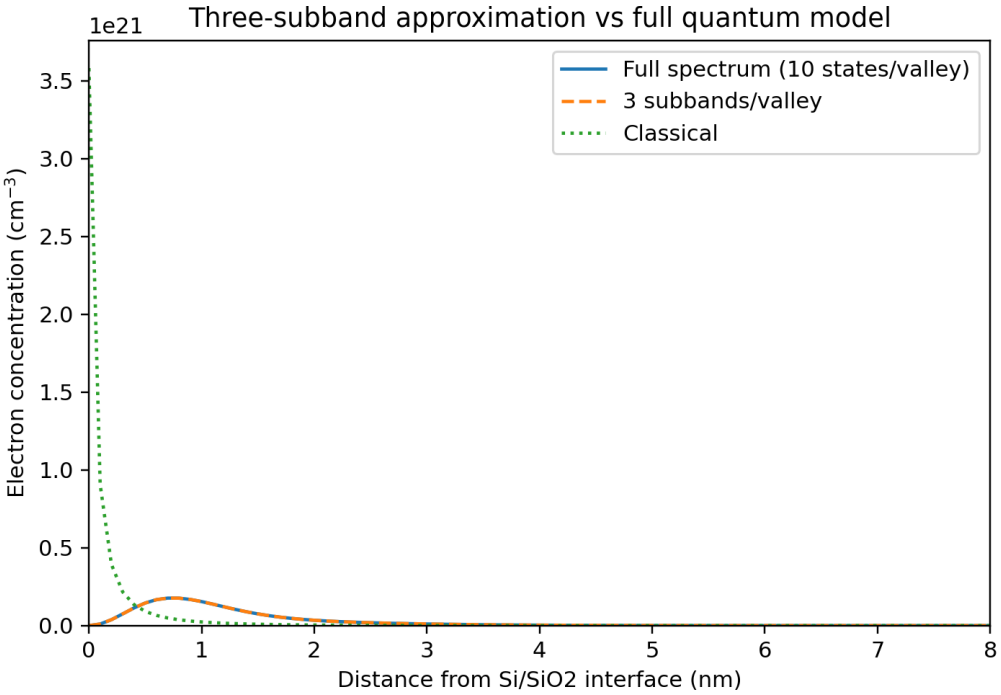


Figure 3: Self-consistent comparison of the ten-state-per-valley reference, the three-subband-per-valley reduced model, and the classical solution. The two quantum curves are nearly indistinguishable on the concentration scale.

10 Relation to the attached paper

Janik and Majkusiak formulate the electrostatics of the two-dimensional electron gas through a self-consistent one-dimensional Schrodinger–Poisson system, solve separate valley families for (100) Si, and compare quantum and classical electron profiles. Their Fig. 3 shows the classical maximum at the interface and the quantum maximum shifted into Si. They also state that rigorous numerical Schrodinger–Poisson analysis is required for accurate surface-potential calculations in ultrathin-oxide MOS structures. The present implementation extends the earlier code by retaining holes and incomplete acceptor ionization throughout the nonlinear Poisson solve. For the reduced-order comparison, it also tests an explicit three-subband truncation. Spinelli et al. independently motivate a three-subband quantum treatment supplemented by a classical high-energy tail; our comparison separates the discrete-state truncation error from that additional tail approximation.

11 Two-dimensional QM-onset map in the t_{ox}–N_A plane

The preceding calculations establish the quantum and classical solutions at individual device points. To answer the more practical question “when does quantum confinement become obvious?”, the same nonlinear Schrodinger–Poisson machinery is swept over oxide thickness and acceptor concentration. This

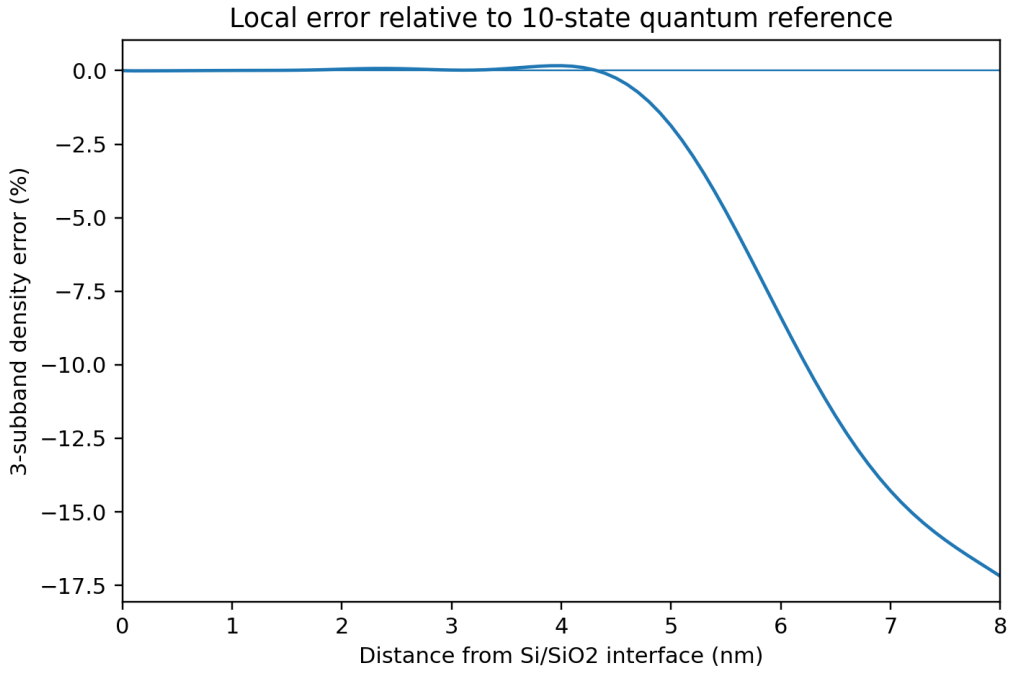


Figure 4: Local electron-density error of the three-subband model relative to the ten-state quantum reference. The relative error can become visually large only where the absolute electron density is already very small; the integrated sheet-charge error remains about -0.0022%.

section is a *model-generated extension*; the contour maps are not digitized from either attached paper. The papers motivate the chosen axes because they identify high substrate doping, strong surface field, and thin gate oxides as the main drivers of inversion-layer quantization [1, 2].

11.1 Sweep domain and operating-point convention

The numerical sweep uses

$$10^{16} \le N_A \le 3 \times 10^{18}\ \mathrm{cm}^{-3}, \qquad 1.5 \le t_{ox} \le 12\ \mathrm{nm}, \tag{27}$$

at $T = 300$ K with the same material parameters, incomplete-ionization model, and exposed flat-band parameter $V_{FB} = -0.95$ V used elsewhere in this report. A six-by-six logarithmic/engineering grid is solved explicitly; the filled contours are interpolation only between those converged grid points.

For the threshold-voltage map, a conventional surface-potential criterion is adopted,

$$\boxed{\phi_s = 2\phi_F, \qquad \phi_F = V_T \ln\!\left(\frac{N_A}{n_i}\right).} \tag{28}$$

At this prescribed ϕ_s, separate classical and quantum semiconductor solutions give the surface electric-field slopes. The gate voltage required to support each solution follows directly from the oxide boundary condition,

$$\boxed{V_{\mathrm{th}} = V_{FB} + \phi_s - \frac{\varepsilon_{Si}}{C_{ox}} \left.\frac{d\phi}{dx}\right|_{x=0}, \qquad C_{ox} = \frac{\varepsilon_{ox}}{t_{ox}}.} \tag{29}$$

This definition intentionally keeps the threshold criterion identical in the quantum and classical calculations so that the voltage shift is caused only by the changed self-consistent semiconductor electrostatics.

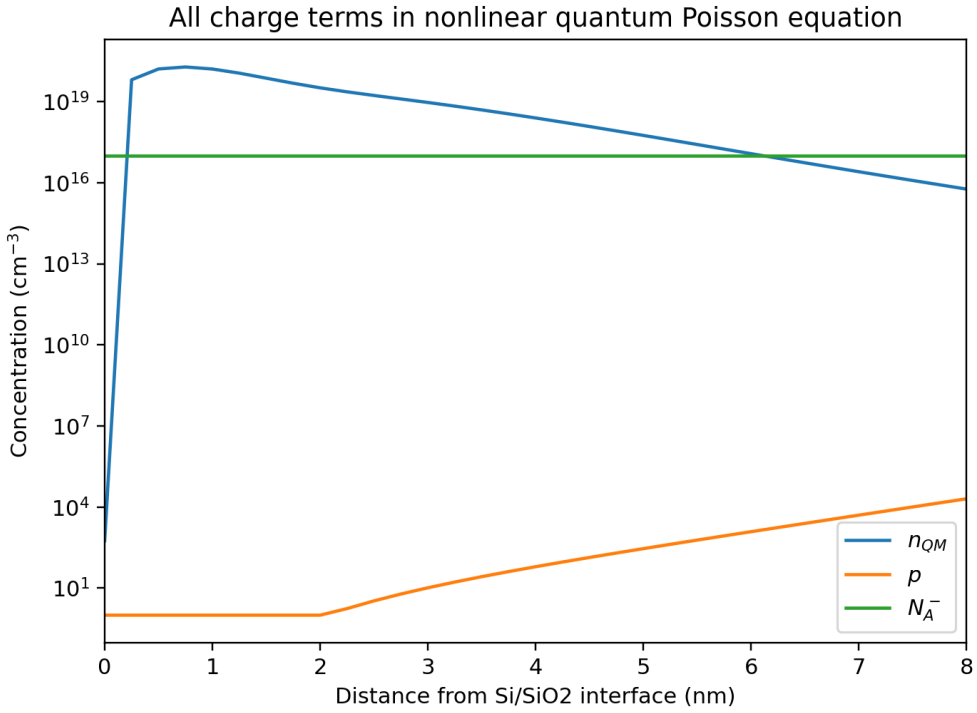


Figure 5: Quantum Poisson charge terms n_{QM}, p, and N_A^-. Holes are strongly suppressed in inversion but are retained rather than set to zero.

The inversion-charge and centroid maps are evaluated one volt above the *classical* threshold at every grid point,

$$\boxed{V_G = V_{\mathrm{th,cl}} + 1\ \mathrm{V}.} \tag{30}$$

Using a threshold-referenced overdrive avoids comparing very different inversion strengths merely because N_A and t_{ox} change the threshold voltage.

11.2 Onset metrics

The inversion sheet charge is computed from the excess electron population,

$$Q_{\mathrm{inv}}^{(M)} = -q \int_0^L [n_M(x) - n_0]\, dx, \qquad M \in \{QM, cl\}, \tag{31}$$

and its quantum correction is

$$\boxed{\delta_Q = 100\, \frac{|Q_{\mathrm{inv}}^{QM} - Q_{\mathrm{inv}}^{cl}|}{|Q_{\mathrm{inv}}^{cl}|}.} \tag{32}$$

The threshold-voltage correction is

$$\boxed{\delta_{V_{\mathrm{th}}} = 100\, \frac{|V_{\mathrm{th}}^{QM} - V_{\mathrm{th}}^{cl}|}{|V_{\mathrm{th}}^{cl}|}.} \tag{33}$$

The inversion-charge centroid is

$$\bar{x}_M = \frac{\int_0^L x\, [n_M(x) - n_0] dx}{\int_0^L [n_M(x) - n_0] dx}. \tag{34}$$

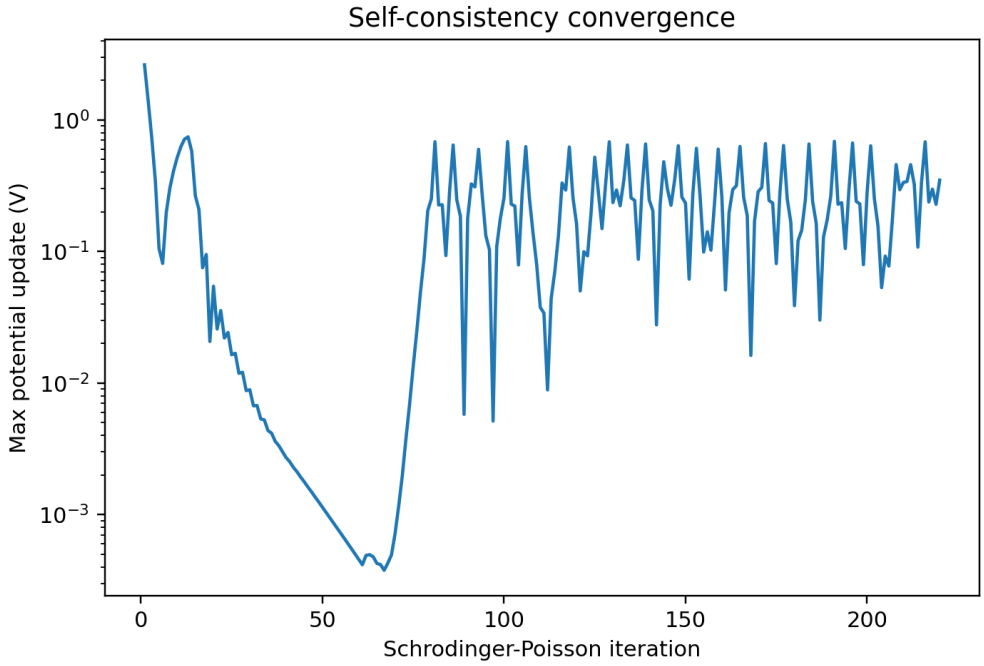


Figure 6: Outer Schrodinger–Poisson convergence history.

A direct relative error $|\bar{x}_{QM} - \bar{x}_{cl}|/\bar{x}_{cl}$ becomes numerically ill-conditioned when the classical centroid approaches the interface. The map therefore uses the physically meaningful oxide-normalized displacement

$$\boxed{\delta_x = 100\,\frac{|\bar{x}_{QM} - \bar{x}_{cl}|}{t_{ox}}.} \tag{35}$$

This quantity is closely related to the effective-oxide-thickness penalty produced by moving inversion charge away from the Si/SiO_2 interface.

11.3 Computed maps and the 1%, 5%, and 10% criteria

Figures 7, 8, and 9 show the requested 1%, 5%, and 10% onset levels wherever those levels are crossed by the computed domain. A requested contour is deliberately omitted when the whole sweep lies on one side of that value; the annotation inside each map states this explicitly rather than inventing an extrapolated contour.

11.4 Interpretation and limits of the onset map

The map demonstrates why a single rule such as "QM matters only for $t_{ox} < 3$ nm" is not generally valid. Oxide thickness changes the gate coupling, whereas doping changes the depletion field and therefore the confinement potential; the quantum response is a coupled function of both. The map should nevertheless be read as a numerical sensitivity study rather than a universal technology boundary. In particular, the absolute values move if V_{FB}, interface charge, the oxide barrier, band-gap narrowing, or the mobility/transport model is changed. The attached papers do not specify all of these quantities, so the present contours are predictions of the explicit model documented here, not additional data attributed to those publications.

For engineering use, the three criteria have different meanings. δ_Q measures electrostatic charge-control error, $\delta_{V_{\rm th}}$ measures a terminal-voltage consequence, and δ_x measures the spatial redistribution that drives effective-oxide-thickness and capacitance changes. A device can therefore exceed 10% in centroid displacement while remaining only a few percent different in threshold voltage. This distinction is physically expected: different terminal observables integrate the same microscopic charge rearrangement in different ways.

Independent confinement-energy check. A useful first-principles companion to the numerical maps is the triangular-well ground-state scale

$$E_1 \simeq a_1 \left(\frac{\hbar^2}{2m_z^*}\right)^{1/3} (qF_s)^{2/3}, \qquad a_1 = 2.3381, \tag{36}$$

and the dimensionless confinement indicator

$$\boxed{\eta_E = E_1/(k_B T).} \tag{37}$$

At a fixed temperature, η_E is controlled primarily by the self-consistent surface field F_s, which is itself set jointly by N_A, t_{ox}, and gate bias. The numerical map and this energy criterion therefore answer complementary questions: the map quantifies terminal/charge errors, whereas η_E identifies whether discrete confinement energies are thermally resolvable.

Indicator	Practical interpretation	Expected modeling consequence
$\eta_E < 1$	weak confinement	classical electrostatics often adequate
$1 \le \eta_E \le 3$	transition region	check $Q_{\rm inv}$, centroid, and $V_{\rm th}$
$\eta_E > 3$	strong confinement	self-consistent QM treatment recommended

Map metric	Computed range	Requested onset contours present
δ_Q	23.9–51.0%	none; entire grid > 10%
$\delta_{V_{\rm th}}$	≈ 0–3.2%	1%
δ_x	7.8–66.2%	10%

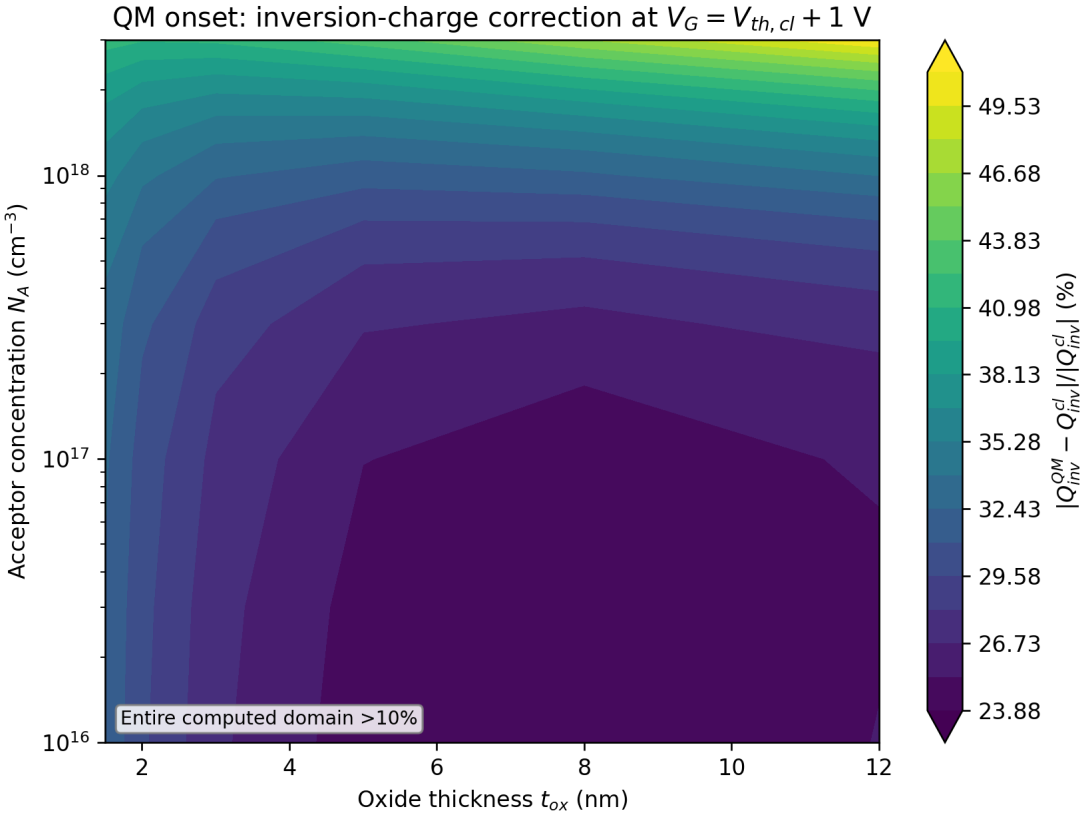


Figure 7: Quantum correction to inversion sheet charge at $V_G = V_{\mathrm{th,cl}} + 1$ V. In the present model and sweep window, δ_Q ranges from approximately 23.9% to 51.0%; consequently the entire computed domain is already beyond the 10% criterion. This strong correction should be interpreted together with the stated model assumptions, especially the flat-band alignment and finite-barrier simplifications.

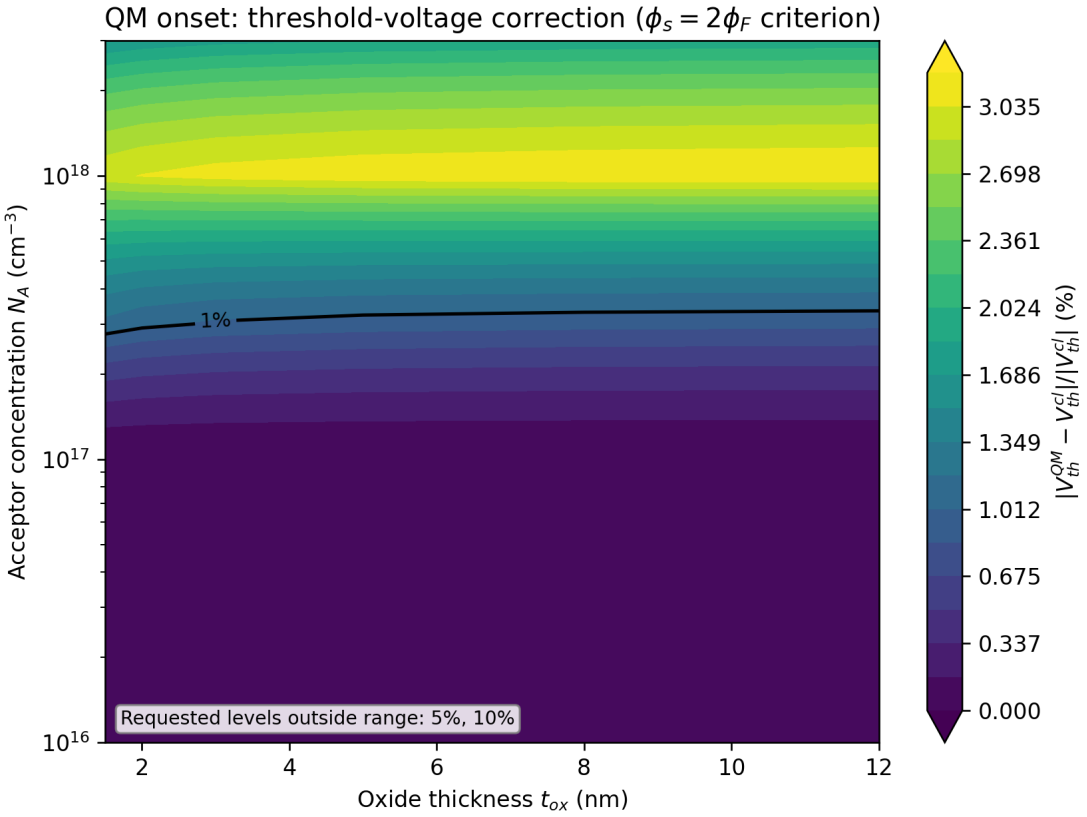


Figure 8: Quantum correction to the threshold voltage using the common $\phi_s = 2\phi_F$ criterion. The computed range is approximately 0–3.2%, so the 1% contour is resolved while the 5% and 10% levels are outside this sweep. The largest relative shift in this parameter window occurs around the $10^{18}\,\mathrm{cm}^{-3}$ doping decade.

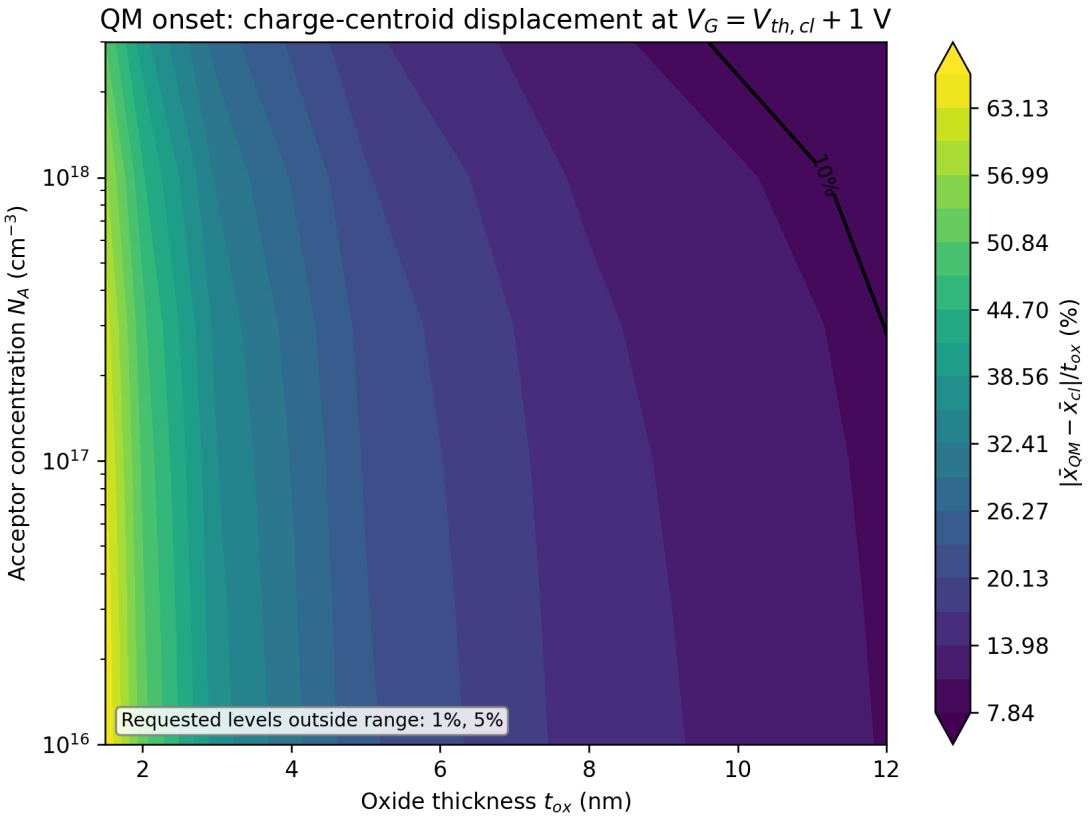


Figure 9: Quantum displacement of the inversion-charge centroid normalized by physical oxide thickness, $\delta_x = 100|\bar{x}_{QM} - \bar{x}_{cl}|/t_{ox}$. The computed range is approximately 7.8%–66.2%; therefore the 10% contour is crossed, while the 1% and 5% criteria lie below almost all of the displayed domain.

A Detailed derivation of the truncated-subband quantum charge

This appendix derives the quantum-density expression used in the three-subband treatment from the one-electron effective-mass Schrödinger equation and the two-dimensional density of states. The notation follows the MOS inversion-layer formulation used by Spinelli *et al.* and is consistent with the two-valley-family implementation used in this report.

A.1 Separation of confined and free motion

Near a planar Si/SiO_2 interface, the electrostatic potential varies rapidly in the direction z normal to the interface and much more slowly in the x–y plane. In the effective-mass approximation, the single-electron Hamiltonian for valley family i can be written

$$\hat{H}_i = -\frac{\hbar^2}{2m^*_{x,i}}\frac{\partial^2}{\partial x^2} - \frac{\hbar^2}{2m^*_{y,i}}\frac{\partial^2}{\partial y^2} - \frac{\hbar^2}{2m^*_{z,i}}\frac{\partial^2}{\partial z^2} + E_c(z). \tag{38}$$

Because E_c depends only on z in the local one-dimensional approximation, the wavefunction separates as

$$\Psi_{ij\mathbf{k}}(x,y,z) = \frac{1}{\sqrt{A}}\exp[i(k_x x + k_y y)]\ \xi_{ij}(z), \tag{39}$$

where A is a normalization area in the interface plane, j labels the quantized subband, and $\mathbf{k} = (k_x, k_y)$ labels the free in-plane state. Substitution into $\hat{H}_i\Psi = E\Psi$ gives the one-dimensional confinement problem

$$\boxed{-\frac{\hbar^2}{2m^*_{z,i}}\frac{d^2\xi_{ij}}{dz^2} + E_c(z)\xi_{ij}(z) = E_{ij}\xi_{ij}(z)} \tag{40}$$

with normalization

$$\int |\xi_{ij}(z)|^2\, dz = 1. \tag{41}$$

The total energy of a state in subband (i,j) is therefore

$$E = E_{ij} + \frac{\hbar^2 k_x^2}{2m^*_{x,i}} + \frac{\hbar^2 k_y^2}{2m^*_{y,i}}. \tag{42}$$

Thus the motion normal to the interface is quantized, whereas the in-plane kinetic energy remains continuous.

A.2 Two-dimensional density of states

For an anisotropic parabolic in-plane dispersion, define the density-of-states mass

$$m^*_{xy,i} = \sqrt{m^*_{x,i}m^*_{y,i}}. \tag{43}$$

Counting the allowed $\mathbf{k}$ states in an area A, including the valley degeneracy g_i, gives a constant two-dimensional density of states per unit area and per unit energy,

$$\boxed{D_i^{2D}(E) = \frac{g_i m^*_{xy,i}}{\pi\hbar^2}H(E - E_{ij})} \tag{44}$$

where H is the Heaviside step function. The factor $1/\pi$ includes the twofold spin degeneracy; equivalently, one may write the spin degeneracy explicitly and use $1/(2\pi)$.

For (100) Si, the six Δ valleys separate into two families. The two valleys whose longitudinal axis is normal to the interface have $g_L = 2$ and confinement mass $m^*_{z,L} = m_l$. The remaining four valleys have $g_T = 4$ and confinement mass $m^*_{z,T} = m_t$. Their in-plane density-of-states masses are evaluated from the appropriate transverse and longitudinal masses.

A.3 Fermi–Dirac population of a complete subband

Let E_{Fn} denote the electron quasi-Fermi level and

$$f(E) = \frac{1}{1 + \exp[(E - E_{Fn})/kT]} \tag{45}$$

be the Fermi–Dirac occupation factor. If the entire subband is treated quantum mechanically, its sheet population is

$$N_{ij} = \int_{E_{ij}}^{\infty} D_i^{2D}(E) f(E)\, dE \tag{46}$$

$$= \frac{g_i m^*_{xy,i}}{\pi\hbar^2} \int_{E_{ij}}^{\infty} \frac{dE}{1 + \exp[(E - E_{Fn})/kT]}. \tag{47}$$

Using

$$\int_a^{\infty} \frac{dE}{1 + \exp[(E - E_F)/kT]} = kT \ln\left[1 + \exp\left(\frac{E_F - a}{kT}\right)\right], \tag{48}$$

we obtain

$$\boxed{N_{ij} = \frac{g_i m^*_{xy,i} kT}{\pi\hbar^2} \ln\left[1 + \exp\left(\frac{E_{Fn} - E_{ij}}{kT}\right)\right]}. \tag{49}$$

Because $|\xi_{ij}(z)|^2$ has dimensions of inverse length, the corresponding three-dimensional local density is

$$n_{ij}(z) = N_{ij} |\xi_{ij}(z)|^2. \tag{50}$$

Summing over all occupied valleys and subbands gives the usual quantum density

$$n_{QM}(z) = \sum_{i,j} N_{ij} |\xi_{ij}(z)|^2. \tag{51}$$

A.4 Truncation at a classical-transition energy

Spinelli *et al.* retain a fixed number of low-lying quantum subbands and represent the higher-energy part of the spectrum classically. Introduce a transition energy E_{cl} above which the electron population is assigned to the classical tail. For each retained subband, only states satisfying

$$E_{ij} \le E < E_{cl} \tag{52}$$

are counted in the explicit quantum contribution. The corresponding sheet population is therefore

$$N_{ij}^{(q)} = \frac{g_i m^*_{xy,i}}{\pi\hbar^2} \int_{E_{ij}}^{E_{cl}} f(E)\, dE. \tag{53}$$

Evaluate the finite Fermi integral by subtracting two upper-tail integrals:

$$\int_{E_{ij}}^{E_{cl}} f(E)\, dE = \int_{E_{ij}}^{\infty} f(E)\, dE - \int_{E_{cl}}^{\infty} f(E)\, dE \tag{54}$$

$$= kT \ln\left[1 + \exp\left(\frac{E_{Fn} - E_{ij}}{kT}\right)\right] - kT \ln\left[1 + \exp\left(\frac{E_{Fn} - E_{cl}}{kT}\right)\right] \tag{55}$$

$$= kT \ln\left[\frac{1 + \exp[(E_{Fn} - E_{ij})/kT]}{1 + \exp[(E_{Fn} - E_{cl})/kT]}\right]. \tag{56}$$

Hence

$$\boxed{N_{ij}^{(q)} = \frac{g_i m_{xy,i}^* kT}{\pi\hbar^2} \ln\left[\frac{1+\exp[(E_{Fn}-E_{ij})/kT]}{1+\exp[(E_{Fn}-E_{cl})/kT]}\right]} \tag{57}$$

and multiplication by the normalized confinement probability yields

$$\boxed{n_q(z) = \frac{kT}{\pi\hbar^2}\sum_{i,j} g_i m_{xy,i}^* \ln\left[\frac{1+\exp[(E_{Fn}-E_{ij})/kT]}{1+\exp[(E_{Fn}-E_{cl})/kT]}\right] |\xi_{ij}(z)|^2}. \tag{58}$$

This is the truncated-subband expression highlighted in the main discussion. The logarithmic ratio is not an empirical correction: it is exactly the Fermi–Dirac population of the energy interval from the subband edge E_{ij} up to the transition energy E_{cl}.

A.5 Classical high-energy tail

The population above E_{cl} is represented with the three-dimensional conduction-band density of states. Under the Maxwell–Boltzmann approximation used for the tail,

$$\boxed{n_t(z) = N_c \exp\left(\frac{E_{Fn} - E_{cl}(z)}{kT}\right)}. \tag{59}$$

The total electron density entering Poisson's equation is then

$$\boxed{n(z) = n_q(z) + n_t(z).} \tag{60}$$

In the three-subband model, the sum in Eq. (58) is restricted to $j = 1, 2, 3$ for each valley family. A practical definition of E_{cl} is the lower of the fourth subband edge and the highest energy that remains bound in the finite quantum domain, matching the truncation philosophy described by Spinelli *et al.*

A.6 Limiting cases and physical interpretation

Several checks are useful when implementing Eq. (58) numerically.

1. If $E_{cl} \to \infty$, the denominator of the logarithm tends to one and Eq. (57) reduces to the complete-subband result in Eq. (49).

2. If $E_{cl} = E_{ij}$, the logarithmic ratio is one, so $N_{ij}^{(q)} = 0$; there is no explicit quantum energy interval left for that subband.

3. If $E_{Fn} \ll E_{ij}$, the Fermi–Dirac expression approaches the Maxwell–Boltzmann limit and the subband population becomes exponentially small.

4. The spatial displacement of inversion charge follows from $|\xi_{ij}(z)|^2$. For a high Si/SiO_2 barrier, the envelope functions are strongly suppressed at the interface, so the quantum density peaks at a finite distance inside Si even when the classical density peaks at the interface.

A.7 Connection to the self-consistent Poisson loop

Equation (58) is evaluated from eigenvalues and eigenfunctions generated by the current electrostatic potential. In the fully coupled model of this report, the Poisson residual is

$$R[\phi] = \frac{d}{dz}\left(\epsilon_{Si}\frac{d\phi}{dz}\right) - q\left[p(\phi) - n(\phi) - N_A^-(\phi)\right], \tag{61}$$

where n may be either the full discrete-state density or the three-subband-plus-tail density of Eq. (60). Consequently, the calculation closes through the chain

$$\phi(z) \longrightarrow E_c(z) \longrightarrow \{E_{ij}, \xi_{ij}\} \longrightarrow n_q + n_t \longrightarrow R[\phi] \longrightarrow \phi_{\mathrm{new}}(z), \tag{62}$$

which is repeated until the electrostatic potential and charge distribution are mutually self-consistent.

B Python implementation

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.linalg import eigh_tridiagonal
from scipy.integrate import solve_bvp
from scipy.optimize import brentq
from scipy.sparse import lil_matrix
from scipy.sparse.linalg import spsolve

# Fully coupled 1-D MOS Schrodinger-Poisson solver
# (100) Si, two valley families, Fermi-Dirac subband occupation,
# holes and incomplete acceptor ionization retained in Poisson equation.

q = 1.602176634e-19
hbar = 1.054571817e-34
kB = 1.380649e-23
m0 = 9.1093837015e-31
eps0 = 8.8541878128e-12

T = 300.0
VT = kB*T/q
eps_si = 11.7*eps0
eps_ox = 3.9*eps0
Eg = 1.122 # eV
NA = 1.0e17*1e6 # m^-3
tox = 3.0e-9 # m
Vg = 4.0 # V
Vfb = -0.95 # V, exposed alignment parameter

# (100) silicon effective masses used in Janik-Majkusiak paper
ml = 0.9163*m0
mt = 0.2245*m0
# name, valley degeneracy, confinement mass, in-plane DOS mass
valleys = [('L', 2, ml, mt), ('T', 4, mt, np.sqrt(ml*mt))]

# Bulk DOS values. Nc from ellipsoidal valley masses; Nv conventional 300 K value.
md3 = (ml*mt*mt)**(1/3)
Nc = 2*6*(md3*kB*T/(2*np.pi*hbar**2))**1.5
Nv = 1.04e19*1e6
ni = np.sqrt(Nc*Nv)*np.exp(-Eg/(2*VT))

# Boron-like acceptor model: acceptor level Ea = Ev + EA_ION.
EA_ION = 0.045 # eV
GA = 4.0 # acceptor degeneracy factor

# Determine equilibrium bulk Fermi level with incomplete ionization.
# eta_v = (Ef-Ev)/kT in volts/eV divided by VT.
def bulk_neutrality(EfEv):
    p = Nv*np.exp(-EfEv/VT)
    n = Nc*np.exp(-(Eg-EfEv)/VT)
    NAminus = NA/(1.0 + GA*np.exp((EA_ION-EfEv)/VT))
    return p - n - NAminus

EfEv = brentq(bulk_neutrality, 0.001, 0.5)
EcEf = Eg - EfEv
p0 = Nv*np.exp(-EfEv/VT)
n0 = Nc*np.exp(-EcEf/VT)
NAOminus = NA/(1.0 + GA*np.exp((EA_ION-EfEv)/VT))
```

```
# Numerical mesh
L = 30e-9
N = 301
x = np.linspace(0.0, L, N)
dx = x[1]-x[0]
Ni = N-2
alpha = eps_ox/(eps_si*tox)


def log1pexp(a):
    a = np.asarray(a, dtype=float)
    out = np.empty_like(a)
    hi = a > 40.0
    lo = a < -40.0
    mid = ~(hi | lo)
    out[hi] = a[hi]
    out[lo] = np.exp(a[lo])
    out[mid] = np.log1p(np.exp(a[mid]))
    return out


def holes(phi):
    """Nondegenerate hole density p(phi), exact bulk value at phi=0."""
    return p0*np.exp(np.clip(-phi/VT, -100.0, 100.0))


def ionized_acceptors(phi):
    """Incomplete ionization N_A^-(phi).

    Electrostatic potential raises Ef-Ev locally by +phi (eV), since both
    semiconductor band edges shift by -q phi while Ef is fixed at equilibrium.
    """
    local_EfEv = EfEv + phi
    arg = np.clip((EA_ION-local_EfEv)/VT, -100.0, 100.0)
    return NA/(1.0 + GA*np.exp(arg))


def classical_electrons(phi):
    """Classical MB electron density for the separate reference solution."""
    return n0*np.exp(np.clip(phi/VT, -100.0, 80.0))


def schrodinger_density(phi, nstates=10):
    """Quantum electron density from two Si valley families.

    Energies are measured relative to the bulk conduction-band edge Ec_bulk.
    U(x) = Ec(x)-Ec_bulk = -phi(x) in eV.
    """
    U = -phi
    ntot = np.zeros_like(x)
    info = []
    for name, g, mz, md in valleys:
        t = (hbar*hbar/(2.0*mz*dx*dx))/q # eV
        diag = U[1:-1] + 2.0*t
        off = -t*np.ones(Ni-1)
        ns = min(nstates, Ni)
        vals, vecs = eigh_tridiagonal(diag, off, select='i', select_range=(0, ns-1))
        sheets = []
        psis = []
        for E, v in zip(vals, vecs.T):
            psi = np.zeros(N)
            psi[1:-1] = v
            psi /= np.sqrt(np.trapezoid(psi*psi, x))
            eta = ((-EcEf)-E)/VT # (Ef - Esub)/kT
            Ns = g*md*kB*T/(np.pi*hbar*hbar)*log1pexp([eta])[0]
            ntot += Ns*psi*psi
            sheets.append(Ns)
            psis.append(psi)
        info.append({'name': name, 'g': g, 'mz': mz, 'md': md,
```

```
                  'E': np.array(vals), 'Ns': np.array(sheets),
                  'psi': np.array(psis)})

    # Add the equilibrium minority density as a smooth far-bulk floor.
    ntot += n0
    return ntot, info


def gate_bc(ya, yb):
    # epsilon_si phi'(0) = Cox[phi_s-(Vg-Vfb)] ; phi(L)=0
    return np.array([ya[1] - alpha*(ya[0]-(Vg-Vfb)), yb[0]])


def solve_poisson_nonlinear(n_fixed, guess, tol=1e-7, maxit=100):
    """Damped-Newton finite-difference Poisson solve.

    Residual contains all local charge terms simultaneously:
      phi'' - q/eps_si [n_fixed + N_A^-(phi) - p(phi)] = 0.
    The quantum density is nonlocal in phi and is refreshed by the outer
    Schrodinger-Poisson iteration.
    """
    M = N-1 # phi[0:M] unknown, phi[N-1]=0 bulk Dirichlet
    ph = guess[:M].copy()
    Vgb = Vg - Vfb
    for _ in range(maxit):
        full = np.zeros(N); full[:M] = ph
        pp = holes(full)[:M]
        naa = ionized_acceptors(full)[:M]
        # derivatives wrt phi
        dpp = -pp/VT
        f = naa/NA
        dna = NA*f*(1.0-f)/VT

        R = np.zeros(M)
        J = lil_matrix((M,M))
        # second-order one-sided gate derivative
        R[0] = (-3*ph[0] + 4*ph[1] - ph[2])/(2*dx) - alpha*(ph[0]-Vgb)
        J[0,0] = -3/(2*dx) - alpha
        J[0,1] = 4/(2*dx)
        J[0,2] = -1/(2*dx)
        for i in range(1,M):
            pim1 = ph[i-1]
            pi = ph[i]
            pip1 = ph[i+1] if i+1<M else 0.0
            R[i] = (pim1 - 2*pi + pip1)/dx**2 - q/eps_si*(n_fixed[i] + naa[i] - pp[i])
            J[i,i-1] = 1/dx**2
            J[i,i] = -2/dx**2 - q/eps_si*(dna[i] - dpp[i])
            if i+1<M: J[i,i+1] = 1/dx**2
        if np.max(np.abs(R[1:])) < 1e10 and abs(R[0]) < 1e3:
            break
        delta = spsolve(J.tocsr(), -R)
        # line search on scaled update; restrict excessively large Newton moves
        maxdu = np.max(np.abs(delta))
        if maxdu > 0.15: delta *= 0.15/maxdu
        lam = 1.0
        norm0 = np.linalg.norm(R)
        accepted = False
        for _ls in range(12):
            trial = ph + lam*delta
            tfull=np.zeros(N); tfull[:M]=trial
            tpp=holes(tfull)[:M]; tna=ionized_acceptors(tfull)[:M]
            Rt=np.zeros(M)
            Rt[0]=(-3*trial[0]+4*trial[1]-trial[2])/(2*dx)-alpha*(trial[0]-Vgb)
            for i in range(1,M):
                pip1=trial[i+1] if i+1<M else 0.0
                Rt[i]=(trial[i-1]-2*trial[i]+pip1)/dx**2-q/eps_si*(n_fixed[i]+tna[i]-tpp[i])
            if np.linalg.norm(Rt) < norm0:
                ph=trial; accepted=True; break
            lam*=0.5
        if not accepted:
```

```
            ph += 0.05*delta
        if np.max(np.abs(lam*delta)) < tol:
            break
    out=np.zeros(N); out[:M]=ph
    return out


def solve_classical(guess=None, tol=1e-7, maxit=150):
    if guess is None:
        guess = 0.9*np.exp(-x/5e-9)
    M=N-1; ph=guess[:M].copy(); Vgb=Vg-Vfb
    for _ in range(maxit):
        full=np.zeros(N); full[:M]=ph
        nn=classical_electrons(full)[:M]
        pp=holes(full)[:M]
        naa=ionized_acceptors(full)[:M]
        dnn=nn/VT; dpp=-pp/VT
        f=naa/NA; dna=NA*f*(1-f)/VT
        R=np.zeros(M); J=lil_matrix((M,M))
        R[0]=(-3*ph[0]+4*ph[1]-ph[2])/(2*dx)-alpha*(ph[0]-Vgb)
        J[0,0]=-3/(2*dx)-alpha; J[0,1]=4/(2*dx); J[0,2]=-1/(2*dx)
        for i in range(1,M):
            pip1=ph[i+1] if i+1<M else 0.0
            R[i]=(ph[i-1]-2*ph[i]+pip1)/dx**2-q/eps_si*(nn[i]+naa[i]-pp[i])
            J[i,i-1]=1/dx**2
            J[i,i]=-2/dx**2-q/eps_si*(dnn[i]+dna[i]-dpp[i])
            if i+1<M: J[i,i+1]=1/dx**2
        delta=spsolve(J.tocsr(),-R)
        maxdu=np.max(np.abs(delta))
        if maxdu>0.10: delta*=0.10/maxdu
        ph += 0.5*delta
        if np.max(np.abs(0.5*delta))<tol: break
    out=np.zeros(N); out[:M]=ph
    return out


# Classical reference
phi_c = solve_classical()
nc = classical_electrons(phi_c)
pc = holes(phi_c)
NAc = ionized_acceptors(phi_c)


def solve_quantum_model(nstates, phi_init=None, max_outer=600):
    """Self-consistent quantum solution retaining nstates per valley family."""
    phi_q = phi_c.copy() if phi_init is None else phi_init.copy()
    n_work, info = schrodinger_density(phi_q, nstates=nstates)
    history = []
    for it in range(max_outer):
        phi_raw = solve_poisson_nonlinear(n_work, phi_q, tol=1e-7)
        nq_new, info = schrodinger_density(phi_raw, nstates=nstates)
        err = np.max(np.abs(phi_raw-phi_q))
        history.append(err)
        beta = 0.01 if it < 100 else 0.02
        floor = max(n0, 1.0)
        logn = (1-beta)*np.log(np.maximum(n_work, floor)) + beta*np.log(np.maximum(nq_new, floor))
        n_work = np.exp(logn)
        phi_q = 0.25*phi_raw + 0.75*phi_q
        dens_err = np.max(np.abs(np.log(np.maximum(nq_new,floor)/np.maximum(n_work,floor))))
        if err < 2e-5 and dens_err < 2e-3:
            break
    nq, info = schrodinger_density(phi_q, nstates=nstates)
    return phi_q, nq, info, np.asarray(history)


# "Full" numerical reference: ten finite-box eigenstates per valley family.
phi_full, nq_full, info_full, hist_full = solve_quantum_model(10)
# Reduced model: only the lowest three subbands in each valley family.
phi_3, nq_3, info_3, hist_3 = solve_quantum_model(3, phi_init=phi_full)
```

```
pq_full = holes(phi_full); NAq_full = ionized_acceptors(phi_full)
pq_3 = holes(phi_3); NAq_3 = ionized_acceptors(phi_3)

# Diagnostics and integrated sheet charge.
def diagnostics(phi, n):
    xmax = x[np.argmax(n)]
    xav = np.trapezoid(x*n, x)/np.trapezoid(n, x)
    Ns = np.trapezoid(np.maximum(n-n0,0.0), x)
    return xmax, xav, Ns

xmax_f,xav_f,Ns_f=diagnostics(phi_full,nq_full)
xmax_3,xav_3,Ns_3=diagnostics(phi_3,nq_3)
rel_Ns=(Ns_3-Ns_f)/Ns_f*100.0
surf_rel=(phi_3[0]-phi_full[0])*1e3

print(f'Bulk Ef-Ev = {EfEv:.6f} eV, Ec-Ef = {EcEf:.6f} eV')
print(f'Full model: phi_s={phi_full[0]:.6f} V, xmax={xmax_f*1e9:.3f} nm, xav={xav_f*1e9:.3f} nm, Ns={Ns_f/1e4:.4e}
     cm^-2')
print(f'3-subband : phi_s={phi_3[0]:.6f} V, xmax={xmax_3*1e9:.3f} nm, xav={xav_3*1e9:.3f} nm, Ns={Ns_3/1e4:.4e} cm
     ^-2')
print(f'3-subband vs full: Delta phi_s={surf_rel:.3f} mV, Delta Ns={rel_Ns:.4f}%')
for tag,info in [('full',info_full),('3sub',info_3)]:
    for a in info:
        print(tag,a['name'],'E [eV rel Ec_bulk]=',np.round(a['E'][:6],5))

# Save comparison data.
rel_profile = 100.0*(nq_3-nq_full)/np.maximum(nq_full,1.0)
out=np.c_[x*1e9,phi_full,phi_3,phi_c,nq_full/1e6,nq_3/1e6,nc/1e6,rel_profile]
np.savetxt('/mnt/data/janik_3subband_comparison.csv',out,delimiter=',',
 header='x_nm,phi_full_V,phi_3sub_V,phi_classical_V,n_full_cm-3,n_3sub_cm-3,n_classical_cm-3,
      relative_density_error_percent',comments='')

m=x<=8e-9
plt.figure(figsize=(6.6,4.8))
plt.plot(x[m]*1e9,nq_full[m]/1e6,label='Full spectrum (10 states/valley)')
plt.plot(x[m]*1e9,nq_3[m]/1e6,'--',label='3 subbands/valley')
plt.plot(x[m]*1e9,nc[m]/1e6,':',label='Classical')
plt.xlabel('Distance from Si/SiO2 interface (nm)')
plt.ylabel('Electron concentration (cm$^{-3}$)')
plt.title('Three-subband approximation vs full quantum model')
plt.xlim(0,8); plt.ylim(bottom=0); plt.legend(); plt.tight_layout()
plt.savefig('/mnt/data/janik_3subband_comparison.png',dpi=220); plt.close()

plt.figure(figsize=(6.6,4.5))
plt.plot(x[m]*1e9,rel_profile[m])
plt.axhline(0,linewidth=0.8)
plt.xlabel('Distance from Si/SiO2 interface (nm)')
plt.ylabel('3-subband density error (%)')
plt.title('Local error relative to 10-state quantum reference')
plt.xlim(0,8); plt.tight_layout()
plt.savefig('/mnt/data/janik_3subband_relative_error.png',dpi=220); plt.close()
```

C MATLAB implementation

```
% Fully nonlinear 1-D Schrodinger-Poisson MOS solver
% Comparison: 10-state reference vs lowest 3 subbands per Si valley family.
clear; clc;
q=1.602176634e-19; hbar=1.054571817e-34; kB=1.380649e-23;
m0=9.1093837015e-31; eps0=8.8541878128e-12;
T=300; VT=kB*T/q; eps_si=11.7*eps0; eps_ox=3.9*eps0;
Eg=1.122; NA=1e17*1e6; tox=3e-9; Vg=4.0; Vfb=-0.95;
ml=0.9163*m0; mt=0.2245*m0; Nv=1.04e19*1e6;
md3=(ml*mt*mt)^(1/3); Nc=2*6*(md3*kB*T/(2*pi*hbar^2))^(3/2);
EA=0.045; gA=4;
fun=@(EfEv) Nv*exp(-EfEv/VT)-Nc*exp(-(Eg-EfEv)/VT)-NA/(1+gA*exp((EA-EfEv)/VT));
EfEv=fzero(fun,[0.001 0.5]); EcEf=Eg-EfEv;
p0=Nv*exp(-EfEv/VT); n0=Nc*exp(-EcEf/VT);
L=30e-9; N=301; x=linspace(0,L,N)'; dx=x(2)-x(1);
alpha=eps_ox/(eps_si*tox);
holes=@(phi) p0.*exp(max(min(-phi/VT,100),-100));
NAminus=@(phi) NA./(1+gA.*exp(max(min((EA-(EfEv+phi))/VT,100),-100)));
nclass=@(phi) n0.*exp(max(min(phi/VT,80),-100));

% Fully nonlinear classical reference.
phi_c=0.9*exp(-x/5e-9);
for it=1:150
    [phi_new,du]=poisson_newton(phi_c,nclass(phi_c),true,x,dx,alpha,Vg,Vfb,q,eps_si,holes,NAminus,nclass,VT,NA);
    phi_c=phi_new; if du<1e-7, break; end
end
nc=nclass(phi_c);

% Quantum 10-state and 3-subband models.
[phi_full,nfull,hfull]=solve_quantum(phi_c,10,x,dx,hbar,q,kB,T,ml,mt,EcEf,n0,alpha,Vg,Vfb,eps_si,holes,NAminus,
     nclass,VT,NA);
[phi_3,n3,h3]=solve_quantum(phi_full,3,x,dx,hbar,q,kB,T,ml,mt,EcEf,n0,alpha,Vg,Vfb,eps_si,holes,NAminus,nclass,VT,
     NA);

Nsfull=trapz(x,max(nfull-n0,0)); Ns3=trapz(x,max(n3-n0,0));
[~,i1]=max(nfull); [~,i3]=max(n3);
fprintf('10-state: phi_s=%.6f V, xmax=%.3f nm, Ns=%.4e cm^-2\n',phi_full(1),x(i1)*1e9,Nsfull/1e4);
fprintf('3-subband: phi_s=%.6f V, xmax=%.3f nm, Ns=%.4e cm^-2\n',phi_3(1),x(i3)*1e9,Ns3/1e4);
fprintf('Delta Ns=%.5f %%\n',100*(Ns3-Nsfull)/Nsfull);

mask=x<=8e-9;
figure; plot(x(mask)*1e9,nfull(mask)/1e6,'LineWidth',1.5); hold on;
plot(x(mask)*1e9,n3(mask)/1e6,'--','LineWidth',1.5);
plot(x(mask)*1e9,nc(mask)/1e6,':','LineWidth',1.5);
xlabel('Distance from Si/SiO2 interface (nm)'); ylabel('Electron concentration (cm^{-3})');
legend('10-state reference','3 subbands/valley','Classical'); grid on;
writematrix([x*1e9 phi_full phi_3 phi_c nfull/1e6 n3/1e6 nc/1e6], 'janik_3subband_comparison_matlab.csv');

function [phi_q,nq,hist]=solve_quantum(phi_init,nstates,x,dx,hbar,q,kB,T,ml,mt,EcEf,n0,alpha,Vg,Vfb,eps_si,holes,
     NAminus,nclass,VT,NA)
phi_q=phi_init; [nwork,~]=qdensity(phi_q,nstates,x,dx,hbar,q,kB,T,ml,mt,EcEf,n0); hist=[];
for it=1:600
    [phi_raw,~]=poisson_newton(phi_q,nwork,false,x,dx,alpha,Vg,Vfb,q,eps_si,holes,NAminus,nclass,VT,NA);
    [nnew,~]=qdensity(phi_raw,nstates,x,dx,hbar,q,kB,T,ml,mt,EcEf,n0);
    err=max(abs(phi_raw-phi_q)); hist(end+1)=err;
    beta=0.01; if it>=100, beta=0.02; end
    floorv=max(n0,1); nwork=exp((1-beta)*log(max(nwork,floorv))+beta*log(max(nnew,floorv)));
    phi_q=0.25*phi_raw+0.75*phi_q;
    if err<2e-5, break; end
end
[nq,~]=qdensity(phi_q,nstates,x,dx,hbar,q,kB,T,ml,mt,EcEf,n0);
end

function [phi,du]=poisson_newton(guess,nfix,isclass,x,dx,alpha,Vg,Vfb,q,eps_si,holes,NAminus,nclass,VT,NA)
N=length(x); M=N-1; ph=guess(1:M); Vgb=Vg-Vfb; du=inf;
for it=1:120
 full=[ph;0]; pp=holes(full); naa=NAminus(full); if isclass, nn=nclass(full); else, nn=nfix; end
 R=zeros(M,1); J=spalloc(M,M,3*M);
```

```
 R(1)=(-3*ph(1)+4*ph(2)-ph(3))/(2*dx)-alpha*(ph(1)-Vgb);
 J(1,1)=-3/(2*dx)-alpha; J(1,2)=4/(2*dx); J(1,3)=-1/(2*dx);
 dpp=-pp/VT; frac=naa/NA; dna=NA.*frac.*(1-frac)/VT; if isclass, dnn=nn/VT; else, dnn=zeros(size(nn)); end
 for i=2:M
   pip1=0; if i<M, pip1=ph(i+1); end
   R(i)=(ph(i-1)-2*ph(i)+pip1)/dx^2-q/eps_si*(nn(i)+naa(i)-pp(i));
   J(i,i-1)=1/dx^2; J(i,i)=-2/dx^2-q/eps_si*(dnn(i)+dna(i)-dpp(i)); if i<M, J(i,i+1)=1/dx^2; end
 end
 d=J\(-R); if max(abs(d))>0.1, d=d*(0.1/max(abs(d))); end
 ph=ph+0.5*d; du=max(abs(0.5*d)); if du<1e-7, break; end
end
phi=[ph;0];
end

function [ntot,levels]=qdensity(phi,nstates,x,dx,hbar,q,kB,T,ml,mt,EcEf,n0)
N=length(x); Ni=N-2; U=-phi; ntot=zeros(N,1); V={2,ml,mt;4,mt,sqrt(ml*mt)}; levels=cell(2,1); VT=kB*T/q;
for iv=1:2
 g=V{iv,1}; mz=V{iv,2}; md=V{iv,3}; t=(hbar^2/(2*mz*dx^2))/q;
 main=U(2:end-1)+2*t; off=-t*ones(Ni-1,1); H=spdiags([off main off],[-1 0 1],Ni,Ni);
 [vec,D]=eigs(H,nstates,'smallestreal'); E=diag(D); [E,ord]=sort(E); vec=vec(:,ord);
 for j=1:nstates
   psi=zeros(N,1); psi(2:end-1)=vec(:,j); psi=psi/sqrt(trapz(x,psi.^2));
   eta=((-EcEf)-E(j))/VT; if eta>40,l=eta; elseif eta<-40,l=exp(eta); else,l=log1p(exp(eta)); end
   Ns=g*md*kB*T/(pi*hbar^2)*l; ntot=ntot+Ns*psi.^2;
 end
 levels{iv}=E;
end
ntot=ntot+n0;
end
```

D Python implementation of the 2-D QM-onset sweep

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.linalg import eigh_tridiagonal
from scipy.optimize import brentq, anderson
from scipy.sparse import lil_matrix
from scipy.sparse.linalg import spsolve
from concurrent.futures import ProcessPoolExecutor, as_completed
import csv, os, warnings
warnings.filterwarnings('ignore')

q=1.602176634e-19; hbar=1.054571817e-34; kB=1.380649e-23
m0=9.1093837015e-31; eps0=8.8541878128e-12
T=300.; VT=kB*T/q; eps_si=11.7*eps0; eps_ox=3.9*eps0; Eg=1.122
ml=0.9163*m0; mt=0.2245*m0
valleys=[('L',2,ml,mt),('T',4,mt,np.sqrt(ml*mt))]
md3=(ml*mt*mt)**(1/3); Nc=2*6*(md3*kB*T/(2*np.pi*hbar**2))**1.5
Nv=1.04e19*1e6; EA_ION=0.045; GA=4.; Vfb=-0.95
L=30e-9; N=101; x=np.linspace(0,L,N); dx=x[1]-x[0]; Ni=N-2
Vg_map=4.0; NSTATES=6
_fd=np.load('/mnt/data/fd_half_table.npz'); FD_ETA=_fd['eta']; FD_F=_fd['F']; FD_D=_fd['dF']

def fdhalf(eta):
    eta=np.asarray(eta); out=np.empty_like(eta,dtype=float)
    lo=eta<FD_ETA[0]; hi=eta>FD_ETA[-1]; mid=~(lo|hi)
    out[lo]=np.exp(eta[lo])
    z=np.maximum(eta[hi],1e-12); out[hi]=4/(3*np.sqrt(np.pi))*z**1.5*(1+np.pi**2/(8*z**2))
    out[mid]=np.interp(eta[mid],FD_ETA,FD_F)
    return out

def dfdhalf(eta):
    eta=np.asarray(eta); out=np.empty_like(eta,dtype=float)
    lo=eta<FD_ETA[0]; hi=eta>FD_ETA[-1]; mid=~(lo|hi)
    out[lo]=np.exp(eta[lo])
    z=np.maximum(eta[hi],1e-12); A=4/(3*np.sqrt(np.pi)); out[hi]=A*(1.5*np.sqrt(z)-0.5*(np.pi**2/8)*z**(-1.5))
    out[mid]=np.interp(eta[mid],FD_ETA,FD_D)
    return out

def log1pexp_scalar(a):
    if a>40: return a
    if a<-40: return np.exp(a)
    return np.log1p(np.exp(a))

def setup_bulk(NAcm):
    NA=NAcm*1e6
    def f(EfEv):
        p=Nv*np.exp(-EfEv/VT); n=Nc*np.exp(-(Eg-EfEv)/VT)
        na=NA/(1+GA*np.exp((EA_ION-EfEv)/VT))
        return p-n-na
    EfEv=brentq(f,0.001,0.7)
    EcEf=Eg-EfEv
    p0=Nv*np.exp(-EfEv/VT); n0=Nc*np.exp(-EcEf/VT)
    return NA,EfEv,EcEf,p0,n0

def model_functions(NAcm,toxnm):
    NA,EfEv,EcEf,p0,n0=setup_bulk(NAcm); tox=toxnm*1e-9
    alpha=eps_ox/(eps_si*tox)
    def holes(phi): return p0*np.exp(np.clip(-phi/VT,-100,100))
    def ion(phi):
        arg=np.clip((EA_ION-(EfEv+phi))/VT,-100,100)
        return NA/(1+GA*np.exp(arg))
    def celec(phi):
        eta=(phi-EcEf)/VT
        return Nc*fdhalf(eta)
    def dcelec(phi):
        eta=(phi-EcEf)/VT
        return Nc*dfdhalf(eta)/VT
    def qdens(phi):
```

```
        U=-phi; nt=np.zeros_like(phi)
        for name,g,mz,md in valleys:
            t=(hbar*hbar/(2*mz*dx*dx))/q
            vals,vecs=eigh_tridiagonal(U[1:-1]+2*t,-t*np.ones(Ni-1),select='i',select_range=(0,NSTATES-1))
            for E,v in zip(vals,vecs.T):
                psi=np.zeros(N); psi[1:-1]=v; psi/=np.sqrt(np.trapezoid(psi*psi,x))
                eta=((-EcEf)-E)/VT
                Ns=g*md*kB*T/(np.pi*hbar*hbar)*log1pexp_scalar(eta)
                nt += Ns*psi*psi
        return nt+n0
    return NA,EfEv,EcEf,p0,n0,alpha,holes,ion,celec,dcelec,qdens

def solve_poisson_fixed_n(nfix,guess,alpha,Vgb,holes,ion,tol=2e-6,maxit=50):
    M=N-1; ph=guess[:M].copy()
    for _ in range(maxit):
        full=np.zeros(N); full[:M]=ph; pp=holes(full)[:M]; naa=ion(full)[:M]
        dpp=-pp/VT; f=naa/(np.max(naa)*0+1) # dummy replaced next line
        # derivative N_A f(1-f)/VT, infer NA from saturation robustly
        # use analytic ratio via local occupancy; NA reconstructed as max ceiling not safe, pass from ion(large phi)
        NAceil=float(ion(np.array([5.0]))[0]); occ=np.clip(naa/NAceil,0,1); dna=NAceil*occ*(1-occ)/VT
        R=np.zeros(M); J=lil_matrix((M,M))
        R[0]=(-3*ph[0]+4*ph[1]-ph[2])/(2*dx)-alpha*(ph[0]-Vgb)
        J[0,0]=-3/(2*dx)-alpha; J[0,1]=4/(2*dx); J[0,2]=-1/(2*dx)
        for i in range(1,M):
            pip1=ph[i+1] if i+1<M else 0.0
            R[i]=(ph[i-1]-2*ph[i]+pip1)/dx**2-q/eps_si*(nfix[i]+naa[i]-pp[i])
            J[i,i-1]=1/dx**2; J[i,i]=-2/dx**2-q/eps_si*(dna[i]-dpp[i])
            if i+1<M: J[i,i+1]=1/dx**2
        delta=spsolve(J.tocsr(),-R); md=np.max(np.abs(delta))
        if md>0.15: delta*=0.15/md
        ph += 0.45*delta
        if np.max(np.abs(0.45*delta))<tol: break
    out=np.zeros(N); out[:M]=ph; return out

def solve_classical_gate(NAcm,toxnm,Vg=Vg_map,guess=None):
    NA,EfEv,EcEf,p0,n0,alpha,holes,ion,celec,dcelec,qdens=model_functions(NAcm,toxnm)
    M=N-1; Vgb=Vg-Vfb
    if guess is None: guess=1.0*np.exp(-x/6e-9)
    ph=guess[:M].copy()
    for _ in range(100):
        full=np.zeros(N); full[:M]=ph; nn=celec(full)[:M]; pp=holes(full)[:M]; naa=ion(full)[:M]
        dnn=dcelec(full)[:M]; dpp=-pp/VT; occ=np.clip(naa/NA,0,1); dna=NA*occ*(1-occ)/VT
        R=np.zeros(M); J=lil_matrix((M,M))
        R[0]=(-3*ph[0]+4*ph[1]-ph[2])/(2*dx)-alpha*(ph[0]-Vgb)
        J[0,0]=-3/(2*dx)-alpha; J[0,1]=4/(2*dx); J[0,2]=-1/(2*dx)
        for i in range(1,M):
            pip1=ph[i+1] if i+1<M else 0
            R[i]=(ph[i-1]-2*ph[i]+pip1)/dx**2-q/eps_si*(nn[i]+naa[i]-pp[i])
            J[i,i-1]=1/dx**2; J[i,i]=-2/dx**2-q/eps_si*(dnn[i]+dna[i]-dpp[i])
            if i+1<M: J[i,i+1]=1/dx**2
        delta=spsolve(J.tocsr(),-R); md=np.max(np.abs(delta))
        if md>0.1: delta*=0.1/md
        ph += 0.55*delta
        if np.max(np.abs(.55*delta))<2e-6: break
    out=np.zeros(N); out[:M]=ph
    return out,celec(out),(NA,EfEv,EcEf,p0,n0,alpha,holes,ion,celec,dcelec,qdens)

def solve_quantum_gate(NAcm,toxnm,Vg=Vg_map):
    pc,nc,mod=solve_classical_gate(NAcm,toxnm,Vg)
    NA,EfEv,EcEf,p0,n0,alpha,holes,ion,celec,dcelec,qdens=mod; Vgb=Vg-Vfb
    count=[0]
    def res(phi):
        count[0]+=1; nq=qdens(np.asarray(phi)); raw=solve_poisson_fixed_n(nq,np.asarray(phi),alpha,Vgb,holes,ion)
        return raw-np.asarray(phi)
    try:
        pq=anderson(res,pc,M=5,maxiter=45,f_tol=2e-4,w0=0.03,verbose=False)
    except Exception as e:
        # NoConvergence stores last iterate as first arg in SciPy
        try: pq=np.asarray(e.args[0],float)
        except Exception: pq=pc.copy()
```

```
    nq=qdens(pq)
    return pc,nc,pq,nq,mod,count[0]

def metrics_gate(NAcm,toxnm,Vg=Vg_map):
    pc,nc,pq,nq,mod,nev=solve_quantum_gate(NAcm,toxnm,Vg)
    n0=mod[4]
    exc_c=np.maximum(nc-n0,0); exc_q=np.maximum(nq-n0,0)
    Ns_c=np.trapezoid(exc_c,x); Ns_q=np.trapezoid(exc_q,x)
    xc=np.trapezoid(x*exc_c,x)/max(Ns_c,1e-300); xq=np.trapezoid(x*exc_q,x)/max(Ns_q,1e-300)
    dQ=100*abs(Ns_q-Ns_c)/max(Ns_c,1e-300)
    dXrel=100*abs(xq-xc)/max(xc,1e-12)
    dX=100*abs(xq-xc)/(toxnm*1e-9) # centroid displacement normalized by physical oxide thickness
    return dQ,dX,dXrel,Ns_c/1e4,Ns_q/1e4,xc*1e9,xq*1e9,pq[0],pc[0],nev

# Threshold voltage via conventional surface-potential criterion phi_s=2 phi_F.
# At fixed phi_s the semiconductor solution depends on NA but not tox; tox only enters Vg=Vfb+phi_s-eps_si/Cox*dphi/
    dx|0.
def threshold_for_NA(NAcm):
    NA,EfEv,EcEf,p0,n0=setup_bulk(NAcm); phiF=VT*np.log(max(NAcm,1)/max(np.sqrt(Nc*Nv)*np.exp(-Eg/(2*VT))/1e6,1e
        -300))
    phis=2*phiF
    # model with arbitrary tox (not used in Dirichlet semiconductor solve)
    mod=model_functions(NAcm,3.0); NA,EfEv,EcEf,p0,n0,alpha,holes,ion,celec,dcelec,qdens=mod
    M=N-2
    def pois_dir(nfix,guess):
        ph=guess[1:-1].copy()
        for _ in range(80):
            full=np.empty(N); full[0]=phis; full[-1]=0; full[1:-1]=ph
            pp=holes(full)[1:-1]; naa=ion(full)[1:-1]; dpp=-pp/VT; occ=np.clip(naa/NA,0,1); dna=NA*occ*(1-occ)/VT
            R=np.zeros(M); J=lil_matrix((M,M))
            for kk,i in enumerate(range(1,N-1)):
                pim1=phis if i==1 else ph[kk-1]; pi=ph[kk]; pip1=0 if i==N-2 else ph[kk+1]
                R[kk]=(pim1-2*pi+pip1)/dx**2-q/eps_si*(nfix[i]+naa[kk]-pp[kk])
                if kk>0: J[kk,kk-1]=1/dx**2
                J[kk,kk]=-2/dx**2-q/eps_si*(dna[kk]-dpp[kk])
                if kk<M-1: J[kk,kk+1]=1/dx**2
            du=spsolve(J.tocsr(),-R); md=np.max(abs(du));
            if md>0.1: du*=0.1/md
            ph += .5*du
            if np.max(abs(.5*du))<2e-6: break
        out=np.empty(N); out[0]=phis; out[-1]=0; out[1:-1]=ph; return out
    # classical Dirichlet
    guess=phis*np.exp(-x/6e-9); ph=guess.copy()
    for _ in range(60):
        nn=celec(ph); new=pois_dir(nn,ph)
        if np.max(abs(new-ph))<2e-5: ph=new; break
        ph=.5*new+.5*ph
    pc=ph
    # quantum Anderson fixed point
    def r(phi): return pois_dir(qdens(np.asarray(phi)),np.asarray(phi))-np.asarray(phi)
    try: pq=anderson(r,pc,M=5,maxiter=40,f_tol=2e-4,w0=.03)
    except Exception as e:
        try: pq=np.asarray(e.args[0],float)
        except: pq=pc.copy()
    # surface derivative (positive x into Si)
    dc=(-3*pc[0]+4*pc[1]-pc[2])/(2*dx); dq=(-3*pq[0]+4*pq[1]-pq[2])/(2*dx)
    # Vg = Vfb + phis - (eps_si/Cox)*phi'(0)
    return phis,dc,dq

def worker(args):
    na,tox,Vg=args
    try:
        vals=metrics_gate(na,tox,Vg); return (na,tox,Vg,*vals,'ok')
    except Exception as e:
        return (na,tox,Vg,*([np.nan]*10),str(e))

if __name__=='__main__':
    NAvals=np.array([1e16,3e16,1e17,3e17,1e18,3e18],float)
    toxvals=np.array([1.5,2.0,3.0,5.0,8.0,12.0],float)
```

```
# First solve the surface-potential-defined threshold state for each doping.
# The threshold criterion used for the map is phi_s = 2 phi_F. The resulting
# semiconductor fields are then translated to V_T for every oxide thickness.
th={}
with ProcessPoolExecutor(max_workers=4) as ex:
    fs={ex.submit(threshold_for_NA,na):na for na in NAvals}
    for f in as_completed(fs):
        th[fs[f]]=f.result(); print('threshold NA',fs[f],th[fs[f]],flush=True)

Vtc_grid=np.zeros((len(NAvals),len(toxvals)))
Vtq_grid=np.zeros_like(Vtc_grid); dVT=np.zeros_like(Vtc_grid)
for i,na in enumerate(NAvals):
    phis,dc,dq=th[na]
    for j,toxnm in enumerate(toxvals):
        Cox=eps_ox/(toxnm*1e-9)
        Vtc=Vfb+phis-(eps_si/Cox)*dc
        Vtq=Vfb+phis-(eps_si/Cox)*dq
        Vtc_grid[i,j]=Vtc; Vtq_grid[i,j]=Vtq
        dVT[i,j]=100*abs(Vtq-Vtc)/max(abs(Vtc),0.05)

# Compare Qinv and charge centroid at a common operating condition:
# one volt above the *classical* threshold at each (NA,tox) point.
tasks=[]
for i,na in enumerate(NAvals):
    for j,tox in enumerate(toxvals):
        tasks.append((na,tox,Vtc_grid[i,j]+1.0))
rows=[]
with ProcessPoolExecutor(max_workers=4) as ex:
    futs=[ex.submit(worker,t) for t in tasks]
    for f in as_completed(futs):
        r=f.result(); rows.append(r); print('done',r[0],r[1],'Vg',r[2],'dQ,dXtox=',r[3],r[4],r[-1],flush=True)

dQ=np.full((len(NAvals),len(toxvals)),np.nan)
dX=np.full_like(dQ,np.nan); dXrel=np.full_like(dQ,np.nan)
extra={}
for r in rows:
    na,tox=r[0],r[1]; i=np.where(NAvals==na)[0][0]; j=np.where(toxvals==tox)[0][0]
    dQ[i,j]=r[3]; dX[i,j]=r[4]; dXrel[i,j]=r[5]; extra[(na,tox)]=r

csvpath='/mnt/data/qm_onset_map_data.csv'
with open(csvpath,'w',newline='') as fh:
    w=csv.writer(fh)
    w.writerow(['NA_cm-3','tox_nm','Vg_eval_V','Vt_classical_V','Vt_quantum_V','dQinv_percent','dVt_percent','
         centroid_shift_over_tox_percent','centroid_relative_change_percent','Ns_classical_cm-2','Ns_quantum_cm
         -2','centroid_classical_nm','centroid_quantum_nm','phi_s_quantum_V','phi_s_classical_V'])
    for i,na in enumerate(NAvals):
        for j,tox in enumerate(toxvals):
            r=extra[(na,tox)]
            # r: na,tox,Vg,dQ,dX,dXrel,Nsc,Nsq,xc,xq,phisq,phisc,nev,status
            w.writerow([na,tox,r[2],Vtc_grid[i,j],Vtq_grid[i,j],dQ[i,j],dVT[i,j],dX[i,j],dXrel[i,j],r[6],r[7],r
                 [8],r[9],r[10],r[11]])

X,Y=np.meshgrid(toxvals,NAvals)
def mapplot(Z,title,cbar,out):
    fig,ax=plt.subplots(figsize=(7.2,5.4))
    zlo=np.nanmin(Z); zhi=np.nanmax(Z)
    levels=np.linspace(zlo,zhi,20) if zhi>zlo else np.linspace(0,1,20)
    cf=ax.contourf(X,Y,Z,levels=levels,extend='both')
    requested=[1,5,10]
    available=[v for v in requested if zlo<=v<=zhi]
    if available:
        cs=ax.contour(X,Y,Z,levels=available,colors='k',linewidths=1.5)
        ax.clabel(cs,fmt=lambda v:f'{v:.0f}%',fontsize=9)
    missing=[v for v in requested if v not in available]
    if missing:
        if zlo>10:
            note='Entire computed domain >10%'
        elif zhi<1:
            note='Entire computed domain <1%'
        else:
```

```
            note='Requested levels outside range: '+', '.join(f'{v}%' for v in missing)
        ax.text(0.02,0.02,note,transform=ax.transAxes,fontsize=8.5,va='bottom',ha='left',bbox=dict(boxstyle='
            round,pad=0.25',facecolor='white',alpha=0.8,edgecolor='0.5'))
    ax.set_yscale('log'); ax.set_xlabel(r'Oxide thickness $t_{ox}$ (nm)'); ax.set_ylabel(r'Acceptor
        concentration $N_A$ (cm$^{-3}$)')
    ax.set_title(title); cb=fig.colorbar(cf,ax=ax); cb.set_label(cbar); fig.tight_layout(); fig.savefig(out,dpi
        =240); plt.close(fig)
mapplot(dQ,r'QM onset: inversion-charge correction at $V_G=V_{T,cl}+1$ V',r'$|Q_{inv}^{QM}-Q_{inv}^{cl}|/|Q_{inv
    }^{cl}|$ (%)','/mnt/data/qm_onset_Qinv.png')
mapplot(dVT,r'QM onset: threshold-voltage correction ($\phi_s=2\phi_F$ criterion)',r'$|V_{th}^{QM}-V_{th}^{cl
    }|/|V_{th}^{cl}|$ (%)','/mnt/data/qm_onset_Vt.png')
mapplot(dX,r'QM onset: charge-centroid displacement at $V_G=V_{T,cl}+1$ V',r'$|\bar{x}_{QM}-\bar{x}_{cl}|/t_{ox}
    $ (%)','/mnt/data/qm_onset_centroid.png')
print('ranges dQ',np.nanmin(dQ),np.nanmax(dQ),'dVT',np.nanmin(dVT),np.nanmax(dVT),'dX/tox',np.nanmin(dX),np.
    nanmax(dX))
```

E References

References

[1] T. Janik and B. Majkusiak, "Analysis of the MOS Transistor Based on the Self-Consistent Solution to the Schrodinger and Poisson Equations and on the Local Mobility Model," *IEEE Transactions on Electron Devices*, vol. 45, no. 6, pp. 1263–1271, June 1998.

[2] A. S. Spinelli, A. Benvenuti, and A. Pacelli, "Self-Consistent 2-D Model for Quantum Effects in n-MOS Transistors," *IEEE Transactions on Electron Devices*, vol. 45, no. 6, pp. 1342–1349, June 1998.